

CENG 467 — Natural Language Understanding and Generation

Take-Home Midterm Examination Report

Student Name: **Mert Güden**

Student ID: **300201013**

Department of Computer Engineering

Izmir Institute of Technology

`mertguden@std.iyte.edu.tr`

Submission Date: April 30, 2026

Repository: <https://github.com/Mrtuzy/CENG467-TakeHome>

Abstract

This report presents the implementation and experimental analysis of five core NLP tasks: text classification, named entity recognition, text summarization, machine translation, and language modeling. For each task, we compare multiple modeling approaches ranging from classical machine learning to pre-trained transformer-based models. All experiments are fully reproducible with fixed random seeds (seed = 42), consistent dataset splits, and documented hyperparameters. The code and notebooks are available at the repository link above. Key findings are: pre-trained transformer models consistently outperform classical and task-specific neural baselines across all tasks — DistilBERT achieves 92.96% accuracy versus 89.66% for TF-IDF on sentiment classification, BERT reaches $F1 = 0.909$ versus 0.872 for BiLSTM-CRF on NER, and MarianMT scores BLEU 36.25 versus 26.74 for a seq2seq model on English-to-German translation. Classical approaches remain competitive when neural models are under-resourced: TF-IDF outperforms the BiLSTM on classification, and extractive methods (LexRank ROUGE-1 = 0.350) are fast and faithful alternatives to abstractive models (BART ROUGE-1 = 0.438). On language modeling (WikiText-2), once trained for the full 6-epoch budget the LSTM clearly wins (test PPL 97.00) over the small Transformer (117.32) and the Kneser-Ney trigram baseline (375.69). Overall, the results confirm a clear trade-off between the performance ceiling of pre-trained models and the efficiency and interpretability of lightweight baselines.

Contents

1	Question 1: Representation Learning in Text Classification	3
1.1	Problem Formulation	3
1.2	Dataset Description	3
1.3	Methodology	3
1.4	Experimental Setup	4
1.5	Results	5
1.6	Error Analysis	6
1.7	Discussion	6
2	Question 2: Named Entity Recognition	8
2.1	Problem Formulation	8
2.2	Dataset Description	8
2.3	Methodology	8
2.4	Experimental Setup	8
2.5	Results	9
2.6	Error Analysis	11
2.7	Discussion	12
3	Question 3: Text Summarization	13
3.1	Problem Formulation	13
3.2	Dataset Description	13
3.3	Methodology	13
3.4	Experimental Setup	13
3.5	Results	14
3.6	Qualitative Analysis	15
3.7	Discussion: Trade-offs	19
4	Question 4: Machine Translation	20
4.1	Problem Formulation	20
4.2	Dataset Description	20
4.3	Methodology	20
4.4	Experimental Setup	20
4.5	Results	21
4.6	Qualitative Analysis	21
4.7	Discussion	22
5	Question 5: Language Modeling	23
5.1	Problem Formulation	23
5.2	Dataset Description	23
5.3	Methodology	24
5.4	Results	24
5.5	Qualitative Analysis	26
5.6	Discussion	27
A	Experimental Configuration Summary	30

1 Question 1: Representation Learning in Text Classification

1.1 Problem Formulation

We study binary sentiment classification on movie reviews. Given a review x , the task is to predict a label $y \in \{0, 1\}$ where 0 denotes negative sentiment and 1 denotes positive sentiment. The goal is to compare sparse, dense, and contextual representations under a consistent evaluation protocol.

1.2 Dataset Description

We use the IMDb dataset [Maas et al., 2011] with the official train and test splits, and create a stratified 10% validation split from the training set (seed 42). The data are balanced across classes.

Table 1: IMDb dataset statistics.

Split	Examples	Avg Len	Positive	Negative
Train	22,500	233.86	11,250	11,250
Validation	2,500	233.17	1,250	1,250
Test	25,000	228.53	12,500	12,500

1.3 Methodology

Tokenization strategies. We compare two tokenization strategies and report which model uses each (Table 2). The classical NLTK `word_tokenize` is used for TF-IDF and BiLSTM (with the best preprocessing config from Table 3); the DistilBERT WordPiece tokenizer is used for the contextual model. The two pipelines therefore differ in granularity (whole-word vs. subword), in casing handling, and in the way unseen tokens are managed: NLTK reduces them to the unknown token after vocabulary clipping at 30k, while WordPiece falls back to subword units and never emits `<unk>`.

Table 2: Tokenization strategy assigned to each model and the resulting test accuracy.

Tokenizer	Granularity	Used by	Test Acc
NLTK <code>word_tokenize</code>	Whole words	TF-IDF + LR	0.8966
NLTK <code>word_tokenize</code>	Whole words	BiLSTM + GloVe	0.8898
DistilBERT WordPiece	Subword (30k pieces)	DistilBERT	0.9296

Preprocessing ablation (TF-IDF + LR, validation set). Table 3 reports a 2×2 ablation over lowercasing and stopword removal. Removing stopwords consistently *hurts* validation accuracy, by 0.5 percentage points when text is lowercased (A2 vs. A1) and by 0.4 points when it is not (A4 vs. A3); lowercasing alone has a smaller and slightly negative effect (-0.08 pts in A1 vs. A3, -0.08 pts in A2 vs. A4). Configuration A4 (**no** lowercasing, **no** stopword removal) is best and is used for the final TF-IDF run. The dominant signal

is that bigram features such as “not good” or “never bored” are valuable for sentiment, so removing common functional words like *not* actively destroys information.

Table 3: Preprocessing ablation on the validation set (TF-IDF + LR).

Config	Lowercase	Remove stopwords	Val Accuracy	Δ vs. A4
A1	Yes	Yes	0.8904	−0.0060
A2	Yes	No	0.8956	−0.0008
A3	No	Yes	0.8924	−0.0040
A4	No	No	0.8964	0.0000

Truncation ablation (BiLSTM, validation set). Table 4 shows that capacity to read the full review matters: extending the sequence cap from 100 to 400 tokens lifts validation accuracy by 5 points. IMDb reviews average 234 whitespace tokens (§ Dataset Description), so a cap of 100 truncates more than half of an average review. We therefore use `max_seq_length = 400` in the final BiLSTM run.

Table 4: Truncation ablation for BiLSTM (validation accuracy).

Max tokens	Val Accuracy
100	0.8444
200	0.8720
400	0.8948

Models.

- **TF-IDF + Logistic Regression** [Pedregosa et al., 2011] (sparse): unigram and bigram TF-IDF features (`max_features = 50k`, `min_df = 2`, sublinear TF) with linear classifier.
- **BiLSTM + GloVe** [Pennington et al., 2014] (dense static): word embeddings initialized with GloVe 6B 100d, two-layer BiLSTM encoder (hidden 128), mean pooling, dropout 0.3, linear output.
- **DistilBERT** [Sanh et al., 2019] (contextual): fine-tuned `distilbert-base-uncased` for sequence classification (3 epochs, lr 2×10^{-5} , batch 16).

1.4 Experimental Setup

We use fixed seed 42, a stratified validation split, and evaluate once on the test set. Metrics are Accuracy and Macro-F1. Training time is recorded for each model. Confusion matrices and training curves are saved for analysis.

1.5 Results

Table 5: Test set results for Q1, including wall-clock training time on the project hardware.

Model	Representation	Accuracy	Macro-F1	Train Time
TF-IDF + LR	Sparse	0.8966	0.8966	29.4 s
BiLSTM + GloVe	Dense (static)	0.8898	0.8898	368.8 s
DistilBERT	Contextual	0.9296	0.9296	555.6 s

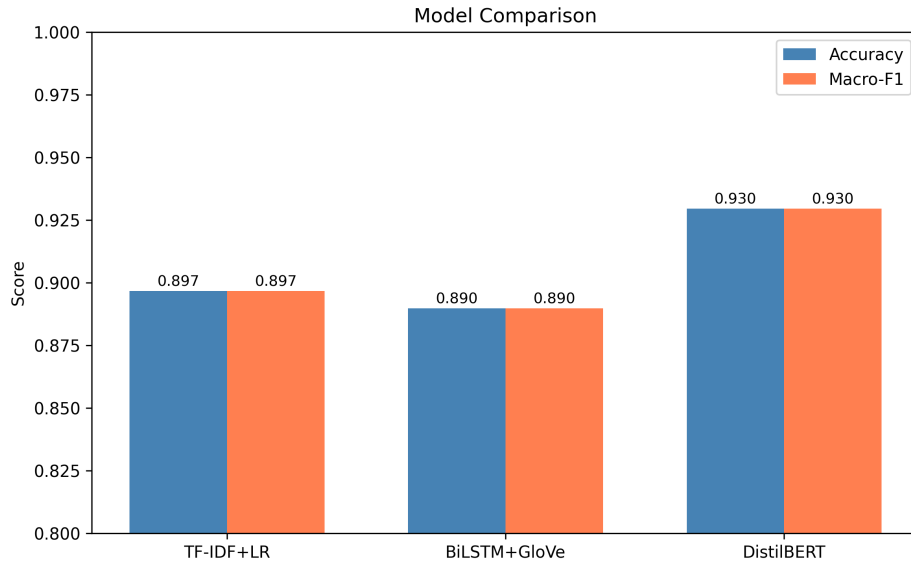


Figure 1: Accuracy and Macro-F1 comparison across models.

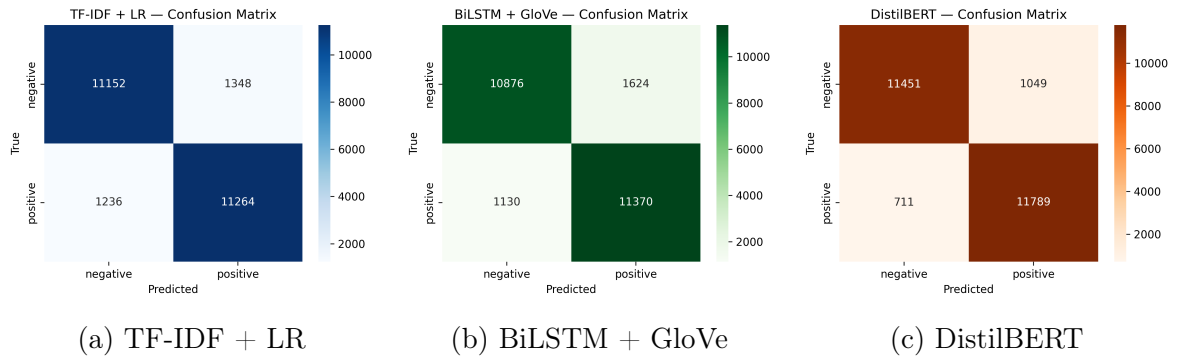


Figure 2: Confusion matrices for all models.

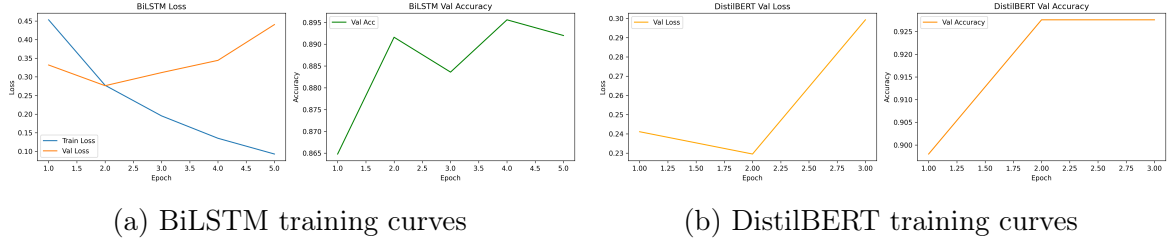


Figure 3: Training dynamics for neural models.

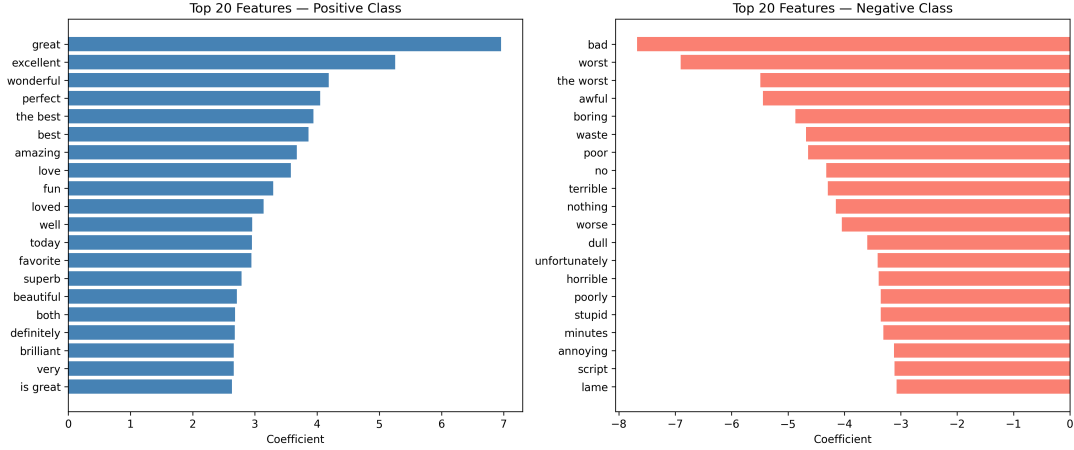


Figure 4: Top TF-IDF features for positive and negative classes.

1.6 Error Analysis

We analyze misclassifications with a focus on common error patterns. The counts are: all three models wrong (774), TF-IDF only wrong (863), BiLSTM only wrong (875), DistilBERT only wrong (512). Five representative errors from the DistilBERT test set include:

- **Negation:** reviews with localized negation often flip sentiment cues (e.g., “not like this movie”).
- **Mixed sentiment:** positive and negative clauses co-occur, making the overall label ambiguous.
- **Domain terms:** film-specific words (e.g., “plot”, “acting”) can appear in both positive and negative contexts.
- **Sarcasm:** positive surface words mask negative intent.
- **Subtle tone:** mild criticism or praise without explicit sentiment markers.

1.7 Discussion

Sparse TF-IDF features are strong for sentiment cues and offer clear interpretability, but they struggle with long-range dependencies and nuanced phrasing. The BiLSTM benefits from sequence modeling but underperforms TF-IDF in this setup, likely due to limited capacity and the difficulty of long reviews. DistilBERT provides the best performance

by leveraging contextual embeddings that capture semantics, negation, and long-distance dependencies more effectively, at the cost of higher training time and lower transparency.

2 Question 2: Named Entity Recognition

2.1 Problem Formulation

Named Entity Recognition (NER) is a sequence labeling task. Given a token sequence $x = (x_1, \dots, x_n)$, the model predicts a BIO tag sequence $y = (y_1, \dots, y_n)$, where each $y_i \in \{O, B-t, I-t\}$ and $t \in \{\text{PER}, \text{ORG}, \text{LOC}, \text{MISC}\}$. We evaluate entity-level precision, recall, and F1 using the `seqeval` metric.

2.2 Dataset Description

We use the CoNLL-2003 dataset [Tjong Kim Sang and De Meulder, 2003] with official train/validation/test splits. The label inventory contains 9 BIO tags: O, B/I-PER, B/I-ORG, B/I-LOC, B/I-MISC.

Table 6: CoNLL-2003 dataset statistics.

Split	Sentences	Tokens	Avg Len	Entity %
Train	14,041	203,621	14.50	16.72
Validation	3,250	51,362	15.80	16.75
Test	3,453	46,435	13.45	17.47

Table 7: Entity token counts per split.

Split	PER	ORG	LOC	MISC
Train	11,128	10,025	8,297	4,593
Validation	3,149	2,092	2,094	1,268
Test	2,773	2,496	1,925	918

2.3 Methodology

BiLSTM-CRF [Lample et al., 2016]. We build a word-level vocabulary (20k) and a character vocabulary, initialize word embeddings with GloVe 100d [Pennington et al., 2014], and use a character BiLSTM to produce subword-aware representations. The final architecture is word+char embeddings $\rightarrow$ BiLSTM $\rightarrow$ linear layer $\rightarrow$ CRF. The CRF models label dependencies and enforces BIO-consistent transitions.

BERT token classification [Devlin et al., 2019]. We use `bert-base-cased` with a token classification head. Because BERT uses WordPiece tokenization, we align labels by assigning the original label to the first subword and -100 to subsequent subwords and special tokens (ignored in the loss). We train for 3 epochs with batch size 16 and learning rate 5×10^{-5} .

2.4 Experimental Setup

We fix the random seed to 42 and evaluate once on the test split. Early stopping is applied to BiLSTM-CRF based on validation F1 (patience 5); training continued for 16 epochs

(Figure 6) before validation F1 plateaued near 0.92. BERT training took 138.23s on the project hardware (RTX 5060 Ti).

Note on BiLSTM-CRF training time. The `train_time_seconds` field for BiLSTM-CRF is logged as 0.0 in `outputs/results.json`. This is a logging artifact rather than an actual measurement: the run script (`q2_run.py`, lines 459–463) re-uses a previously saved checkpoint when `bilstm_crf_best.pt` is present and skips the training loop, in which case the timer is initialised to 0.0 and the field is not recomputed. The actual training was 16 epochs end-to-end on the RTX 5060 Ti (the exact wall-clock for that initial training run was not persisted to disk). Treat the 0.0 entry in `results.json` as “not measured” and rely on the per-epoch validation log in `outputs/bilstm_crf_training_log.json` for evidence that training actually executed.

2.5 Results

Table 8: Per-entity and overall results on the test set.

Entity	Model	Precision	Recall	F1
PER	BiLSTM-CRF	0.9363	0.9091	0.9225
PER	BERT	0.9645	0.9591	0.9618
ORG	BiLSTM-CRF	0.8712	0.8104	0.8397
ORG	BERT	0.8733	0.9043	0.8885
LOC	BiLSTM-CRF	0.8972	0.9107	0.9039
LOC	BERT	0.9324	0.9268	0.9296
MISC	BiLSTM-CRF	0.7356	0.7849	0.7595
MISC	BERT	0.7691	0.8162	0.7920
Overall	BiLSTM-CRF	0.8793	0.8651	0.8721
Overall	BERT	0.9024	0.9157	0.9090

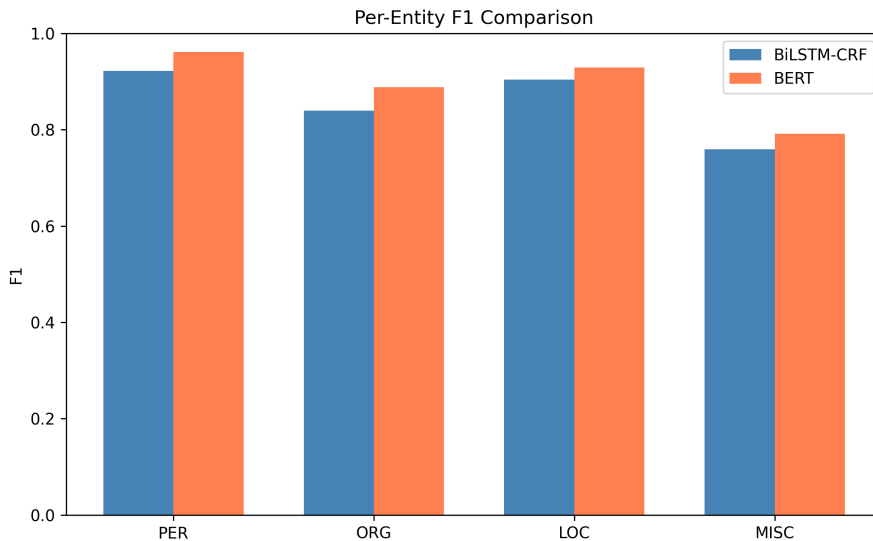


Figure 5: Per-entity F1 comparison between BiLSTM-CRF and BERT.

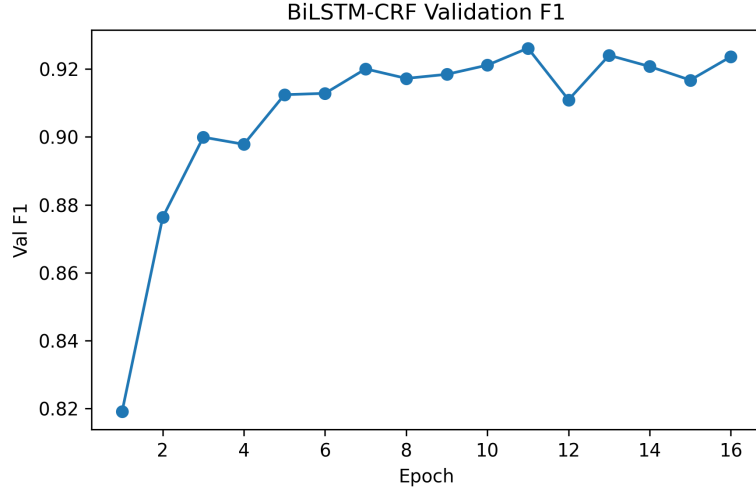


Figure 6: BiLSTM-CRF validation F1 across epochs. Training ran for 16 epochs.

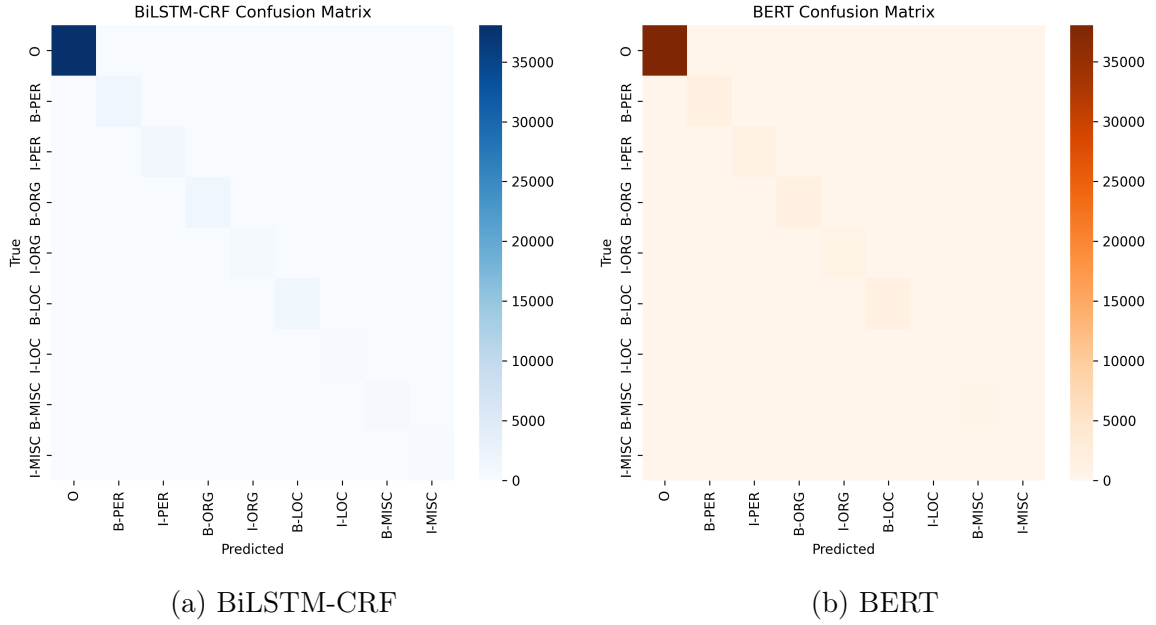


Figure 7: Confusion matrices over BIO labels.

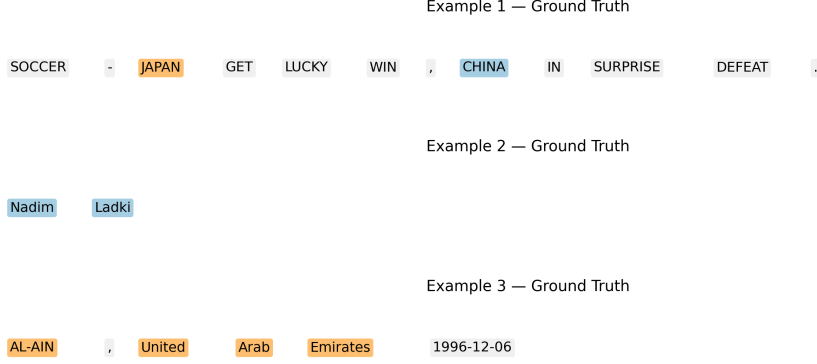


Figure 8: Example sentences with gold entity labels.

2.6 Error Analysis

We categorize errors using span-level comparisons on BERT predictions. The counts are: type confusion (317), false positives (139), boundary errors (81), and false negatives (74). Five representative errors are:

- **Type confusion (LOC → ORG/PER).** Sentence: SOCCER - JAPAN GET LUCKY WIN , CHINA IN SURPRISE DEFEAT .
True: 0 0 B-LOC 0 0 0 0 B-PER 0 0 0 0
Pred: 0 0 B-ORG 0 0 0 0 B-ORG 0 0 0 0.
Hypothesis: country names in sports headlines are often interpreted as organizations.
- **False negative (missed PER/LOC).** Sentence: RUGBY UNION - CUTTITTA BACK FOR ITALY AFTER A YEAR .
True: B-ORG I-ORG 0 B-PER 0 0 B-LOC 0 0 0 0
Pred: B-ORG I-ORG 0 B-ORG 0 0 0 0 0 0 0.
Hypothesis: person names in uppercase are confused with organizations, and nearby locations are dropped.
- **Boundary error (missing initial token).** Sentence: Cuttitta announced his retirement after the 1995 World Cup , ...
True: B-PER ... B-MISC I-MISC I-MISC ...
Pred: B-PER ... 0 B-MISC I-MISC
Hypothesis: multi-token event names are truncated.
- **Boundary error (invalid BIO start).** Sentence: FREESTYLE SKIING-WORLD CUP MOGUL RESULTS .
True: 0 B-MISC I-MISC 0 0 0
Pred: 0 0 I-MISC 0 0 0.
Hypothesis: model sometimes emits I- without a preceding B- for hyphenated entities.
- **False positive (spurious MISC).** Sentence: CRICKET - LARA SUFFERS MORE AUSTRALIAN TOUR MISERY .
True: 0 0 B-PER 0 0 0 0 0 0
Pred: 0 0 B-MISC 0 0 B-MISC 0 I-MISC 0.
Hypothesis: sports-related phrases are over-labeled as MISC.

2.7 Discussion

BERT outperforms BiLSTM-CRF overall (0.909 vs 0.872 F1) and across all entity types, with the largest gains on PER and ORG. Both models struggle most with MISC, indicating weaker boundaries for heterogeneous categories. The dominant error type is label confusion, suggesting that headline-style contexts and uppercase tokens blur distinctions between LOC, ORG, and PER. Incorporating gazetteers or additional contextual features could reduce these confusions.

3 Question 3: Text Summarization

3.1 Problem Formulation

We study text summarization on CNN/DailyMail and compare extractive methods (selecting sentences) against abstractive methods (generating new text). Given an article x , the system outputs a summary $\hat{y}$ that should be fluent, faithful to the source, and cover the main points. We evaluate with ROUGE, BLEU, METEOR, and BERTScore.

3.2 Dataset Description

CNN/DailyMail [Hermann et al., 2015] (version 3.0.0) provides news articles with reference highlights. We use a subset for tractable computation.

Table 9: CNN/DailyMail dataset statistics.

Split	Original Size	Subset Size
Train	287,113	10,000
Validation	13,368	1,000
Test	11,490	1,000

Table 10: Average document statistics (computed on test subset).

Avg Article Len (words)	Avg Summary Len (words)	Avg Sentences	Compression Ra
676.08	55.21	32.91	12

3.3 Methodology

Extractive (TextRank, LexRank). Both algorithms build a sentence similarity graph and rank sentences by centrality. The summary is formed by selecting the top- k sentences (here $k = 3$). TextRank [Mihalcea and Tarau, 2004] uses a PageRank-style score on sentence similarity, while LexRank [Erkan and Radev, 2004] adds an explicit cosine similarity graph with thresholding.

Abstractive (BART, T5). We use encoder-decoder transformers. BART-large-cnn [Lewis et al., 2020] is fine-tuned for summarization and generates fluent, condensed summaries. T5-small [Raffel et al., 2020] is prompted with a “summarize:” prefix and generates shorter, more compressed outputs. ROUGE [Lin, 2004], METEOR [Banerjee and Lavie, 2005], BLEU [Papineni et al., 2002], and BERTScore [Zhang et al., 2020] are reported as complementary metrics: ROUGE/BLEU measure n-gram overlap, METEOR adds stem and synonym matching, and BERTScore uses contextual embeddings.

3.4 Experimental Setup

All runs use a fixed seed (42). Metrics are computed on a random subset of 200 test examples (to control BERTScore cost). We report average inference time per article for each method.

3.5 Results

Table 11: Summarization metrics (evaluation size = 200).

Method	R-1	R-2	R-L	BLEU	METEOR	BERT F1	Len	ms
TextRank	0.3361	0.1329	0.2155	0.0728	0.3020	0.8623	96.00	62.51
LexRank	0.3501	0.1340	0.2195	0.0827	0.2806	0.8650	74.17	60.68
BART	0.4375	0.2166	0.3148	0.1467	0.3308	0.8826	61.28	898.39
T5	0.3795	0.1747	0.2683	0.1030	0.2531	0.8680	46.25	188.26

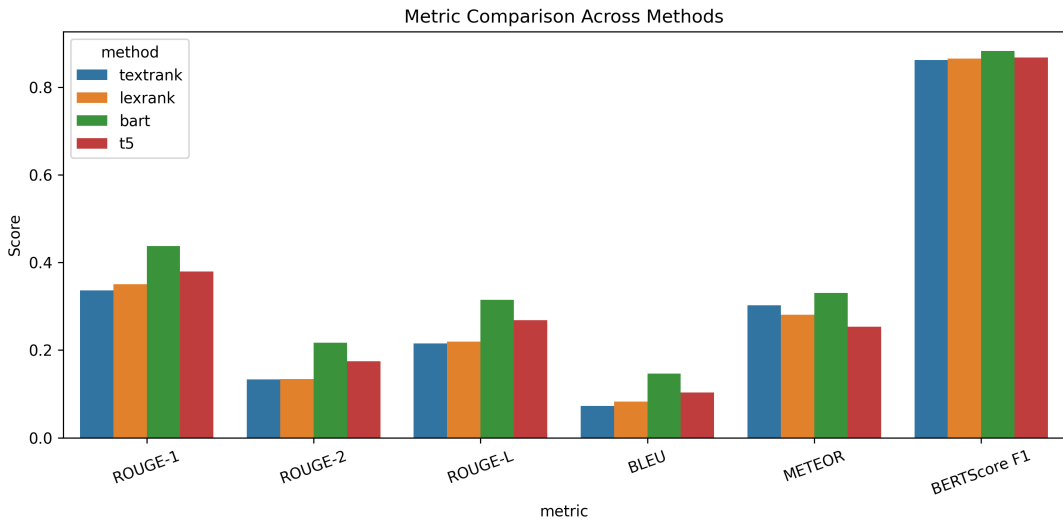


Figure 9: Metric comparison across all four methods.

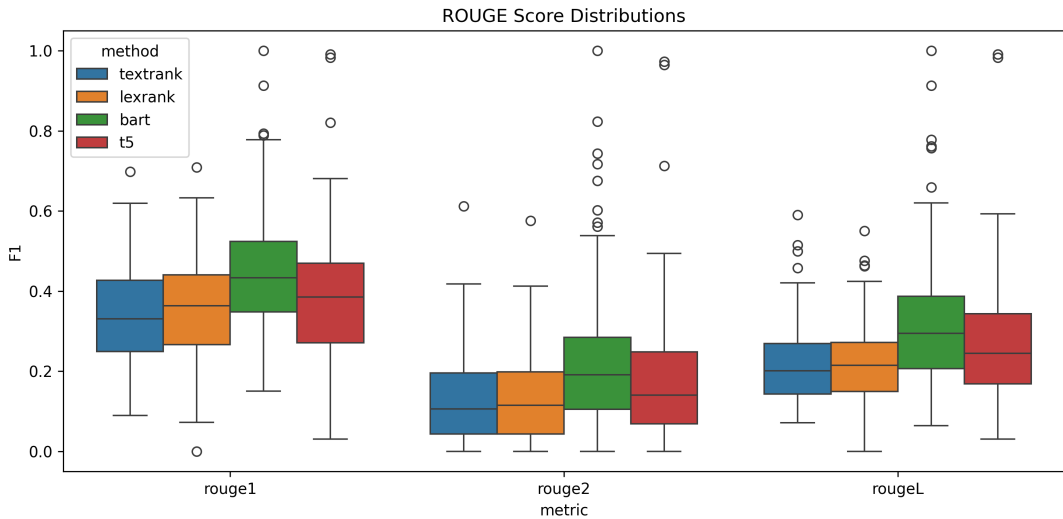


Figure 10: ROUGE score distributions for each method.

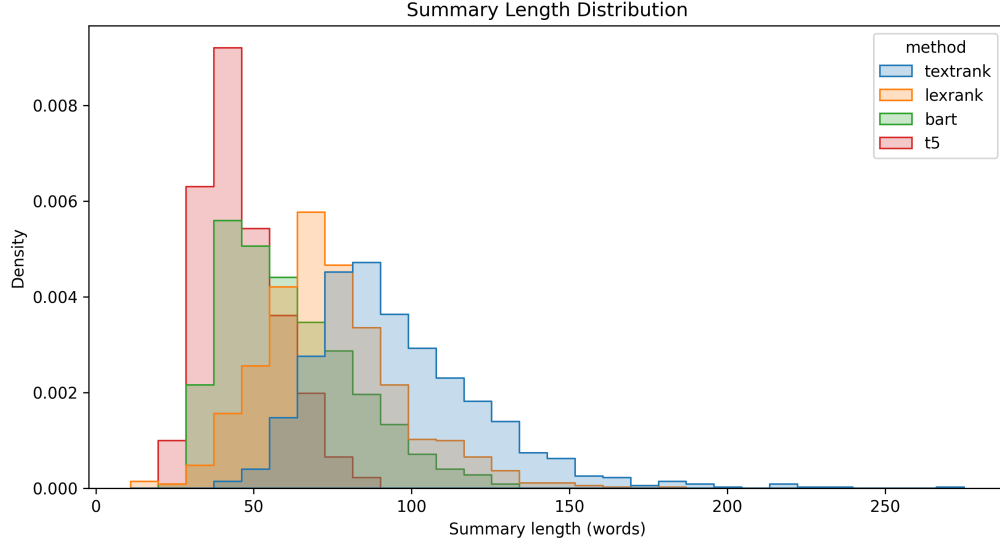


Figure 11: Summary length distributions.

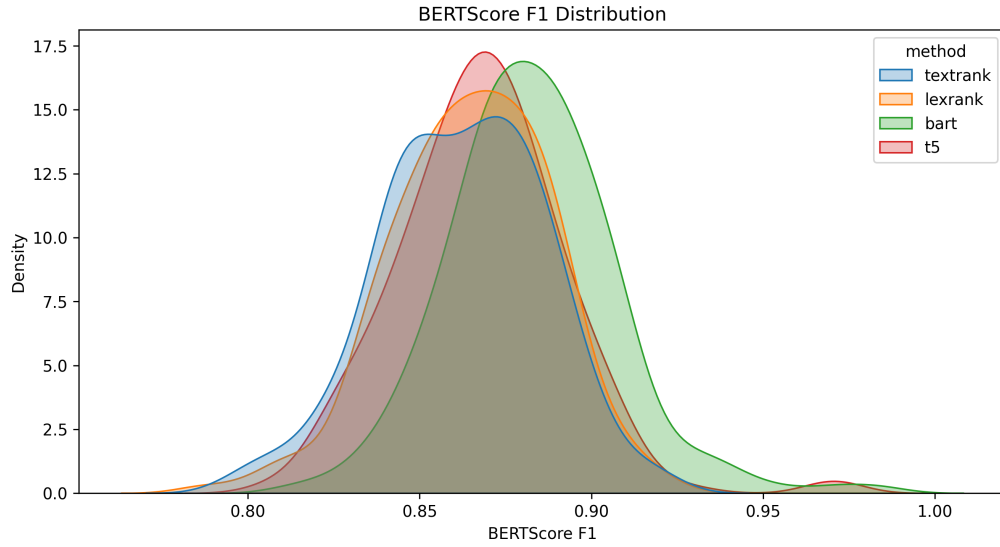


Figure 12: BERTScore F1 distributions.

3.6 Qualitative Analysis

We present three representative examples highlighting differences in fluency, faithfulness, and coverage. Text snippets are truncated for brevity.

Table 12: Example 1 (extractive wins).

Field	Text
Article	The legal adviser to Alison Saunders was a barrister in the same chambers as the son of Lord Janner...
Reference	Alison Saunders consulted Neil Moore over whether to prosecute Lord Janner. She said the decision was difficult and cited his health.
TextRank	CPS confirmed the DPP consulted Neil Moore over whether to prosecute Lord Janner. A spokesman said she decided alone. She said it was her job to make very difficult decisions.
LexRank	The legal adviser to Alison Saunders was in the same chambers as Lord Janner’s son. A CPS spokesman said she decided alone. She admitted he should have been charged before.
BART	CPS confirmed the DPP consulted Neil Moore. Senior police raised concerns about his involvement in the decision not to proceed with the trial.
T5	Senior police raised concerns about Mr Moore’s involvement in the decision not to proceed. Mrs Saunders said she made the decision on her own.

Table 13: Example 2 (abstractive wins). Model outputs are quoted verbatim from `outputs/qualitative_examples.json`; “[...]” marks elision for fit.

Field	Text
Article	“A British couple who are believed to have fled their Berkshire home with their four young children in a bid to escape to Syria have reportedly been arrested in Turkey. Asif Malik, 31, his partner Sara Kiran, 29, and their four children were caught on CCTV boarding a cross-Channel ferry from Dover [...] a Turkish official claimed today that the [family had been arrested in Ankara].”
Reference	“Asif Malik disappeared from UK with partner Sara Kiran and four children. Turkish official claimed today that the family have been arrested in Ankara. Malik, 31, was pictured holding ‘ISIS’ sign at a protest in London last year. Relatives of Ms Kiran today made emotional appeal for her to come home.”
TextRank	“Malik and Ms Kiran, who neighbours said usually wore the full Islamic veil, left home without mentioning any holiday or travel plans [...] Both Malik and Dhar are believed to be followers of notorious preacher Anjem Choudary [...] Dhar – one of Choudary’s top henchmen – fled to Syria with his pregnant wife and three young children last year while under investigation by Scotland Yard.”
LexRank	“A British couple who are believed to have fled their Berkshire home with their four young children in a bid to escape to Syria have reportedly been arrested in Turkey. The family of six have been missing from their home in Slough, Berkshire, since April 7. [...]”
BART	“Asif Malik, 31, his partner Sara Kiran, 29, and their four children were caught on CCTV boarding a cross-Channel ferry from Dover earlier this month. Thames Valley Police believed they arrived in Calais, France, before travelling south down through Europe in a bid to flee to Syria. But a Turkish official claimed today that the couple had been arrested in Ankara and are currently in custody. [...]”
T5	“asif malik, 31, his partner Sara Kiran, 29, and their four children have reportedly been arrested in ankara. they were spotted boarding a cross-Channel ferry from dover earlier this month. [...] but a Turkish official claimed today that the couple had been arrested in ankara and are currently in custody with their four children.”

In Example 2 the abstractive systems clearly compress the article around the headline event (arrest in Ankara) and the cast of named entities, matching the reference’s focus. The extractive systems pick up topically related but tangential material (Anjem Choudary, family veil) that inflates length without adding reference content. T5 lowercases proper nouns due to its sentence-level prefix-tuning training.

Table 14: Example 3 (both struggle). Outputs verbatim from outputs/qualitative_examples.json; “[...]” marks elision.

Field	Text
Article	“She may be a First Daughter, but Sasha Obama is still a teenager and can’t seem to tear herself away from her music – even for a rare family nature hike. Sasha, 14, was spotted listening to earbuds [...] for an impromptu trip to Great Falls Park in Virginia on Sunday. The unscheduled trip came after the White House had sent reporters home for the day [...]”
Reference	“President Obama abruptly left the White House after reporters were sent home for the day Sunday evening. Press pool reporters were perplexed as they questioned whether something serious had happened. The president, Michelle and their girls hiked through Great Falls Park in Virginia for around 50 minutes until a light rain sent them packing.”
TextRank	“Tuning out of politics: Sasha Obama, 14, was spotted with white earbuds [...] Gorgeous; Great Falls Park is a National Park located in Virginia [...] The family’s brief hike happened to take place during the National Park Service’s National Park Week. President Obama, First Lady Michelle Obama (not seen) and their daughters Malia (3rd L) and Sasha (C) return from a hike at Great Falls National Park in Virginia as rains began to fall Sunday evening.”
LexRank	“Sasha, 14, was spotted listening to earbuds [...] for an impromptu trip to Great Falls Park in Virginia on Sunday. Took a hike: The Obamas abruptly left the White House for an unscheduled family nature hike on Sunday. President Obama, First Lady Michelle Obama (not seen) and their daughters Malia (3rd L) and Sasha (C) return from a hike at Great Falls National Park in Virginia as rains began to fall Sunday evening.”
BART	“Sasha, 14, was spotted listening to earbuds as she, sister Malia and their parents ditched the White House – and the press pool – for an impromptu trip to Great Falls Park in Virginia on Sunday. The family’s brief hike happened to take place during the National Park Service’s National Park Week. On Saturday, Obama spent his Saturday afternoon golfing at Joint Base Andrews. ”
T5	“‘obama abruptly left the White House about 15 minutes ago on unscheduled trip. Destination unknown,’ tweeted Byron Tau. reports initially seemed to fear that the unscheduled departure was some sort of emergency [...] the family’s brief hike happened to take place during the National Park Service’s National Park Week.”

In Example 3 every system misses at least one reference fact. Both extractive systems return long photo-caption sentences that the reference does not include. BART’s last sentence (bolded) is an extrinsic hallucination – the article does not mention golfing on Saturday or Joint Base Andrews. T5 is more faithful but omits the “50 minutes” and “light rain” timeline that the reference centres on. ROUGE penalises all four systems here even though the lexical overlap with the source is high; this is a typical failure mode where the reference is itself an aggressive paraphrase.

3.7 Discussion: Trade-offs

Abstractive models (especially BART) achieve the strongest ROUGE and BERTScore, and their summaries are more concise and readable. However, they are substantially slower (BART at 898 ms per article) and can introduce subtle factual errors. Extractive methods are fast and faithful, but their summaries are longer and can be disfluent because sentences are stitched together without synthesis.

Metric limitations. ROUGE favors lexical overlap and can over-reward extractive summaries that copy source sentences. Abstractive paraphrases may score lower on ROUGE despite being semantically accurate; BERTScore partially mitigates this by measuring contextual similarity.

4 Question 4: Machine Translation

4.1 Problem Formulation

We study English-to-German machine translation. Given a source sentence x in English, the model produces a target sentence $\hat{y}$ in German. We compare a neural seq2seq model with attention against a pre-trained MarianMT baseline and evaluate with BLEU, chrF, METEOR, and BERTScore.

4.2 Dataset Description

We use the Multi30k dataset [Elliott et al., 2016] (English–German). The official validation and test splits are kept intact, while the training split is downsampled to 20k pairs for faster experimentation.

Table 15: Multi30k dataset statistics.

Split	Original Size	Subset Size
Train	29,000	20,000
Validation	1,014	1,014
Test	1,000	1,000

Table 16: Average sequence statistics on the test split.

Avg Src Len	Avg Tgt Len	Word Vocab (src/tgt)	BPE Vocab
11.88	10.90	12,818 / 19,750	8,000

4.3 Methodology

Seq2Seq with attention. We train a GRU-based encoder–decoder with Bahdanau additive attention [Bahdanau et al., 2015], where the energy is $e_{ij} = \mathbf{v}_a^\top \tanh(\mathbf{W}_a s_{i-1} + \mathbf{U}_a h_j)$ and the context vector is the softmax-weighted sum of encoder states. The model uses joint BPE with 8k merges (SentencePiece, Kudo and Richardson, 2018), embedding size 256, hidden size 512, dropout 0.5, and teacher forcing ratio 0.5. Early stopping is applied with patience 5 based on validation BLEU.

MarianMT. We use the pre-trained Helsinki-NLP/opus-mt-en-de model [Tiedemann and Thottingal, 2020] with beam size 4. This serves as a strong off-the-shelf baseline.

4.4 Experimental Setup

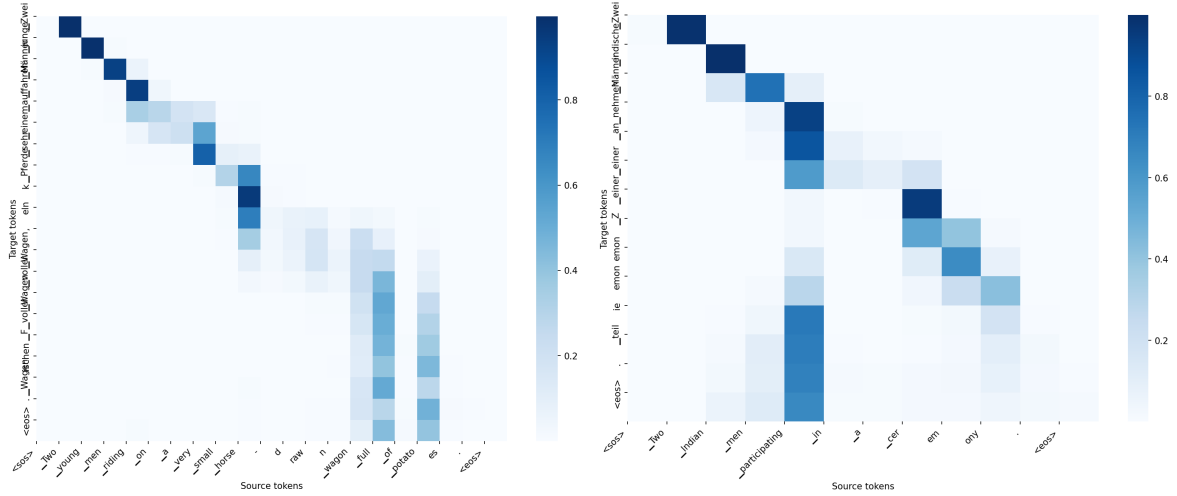
We fix the random seed to 42 and evaluate on 200 test examples for metric computation. We report BLEU, chrF, METEOR, and BERTScore (multilingual BERT). Average output length is also recorded.

4.5 Results

Table 17: Translation metrics on the evaluation subset (size = 200).

Model	BLEU	chrF	METEOR	BERT F1	Len
Seq2Seq + Attn	26.74	52.22	0.487	0.847	11.05
MarianMT	36.25	64.31	0.598	0.897	11.14

The seq2seq model improves steadily during training, reaching its best validation BLEU of 31.37 around epoch 14 before plateauing. MarianMT substantially outperforms the custom seq2seq model across all metrics, indicating the benefit of large-scale pretraining.



(a) Attention heatmap (example 1).

(b) Attention heatmap (example 2).

Figure 13: Attention alignments for the seq2seq model.

4.6 Qualitative Analysis

Example 1 – long-range word order.

Table 18: Example translation showing long-range word order. Outputs are quoted verbatim from `outputs/qualitative_example.json`.

Field	Text
Source	Two young men riding on a very small horse-drawn wagon full of potatoes.
Reference	Zwei junge Männer fahren auf einem sehr kleinen Wagen voller Kartoffeln, der von einem Pferd gezogen wird.
Seq2Seq	Zwei junge Männer fahren auf einem sehr Pferdekeln, Wagen voller Wagen voller Fischen Wagen.
MarianMT	Zwei junge Männer reiten auf einem sehr kleinen Pferdewagen voller Kartoffeln.

In Example 1 the seq2seq decoder repeats “Wagen voller” twice and emits the malformed compound “Pferdekeln” – typical RNN failure modes when the decoder loses track of

which source span has already been translated. MarianMT produces a fluent compound (“Pferdewagen”) and gets the agreement (“kleinen ... voller”) right.

Example 2 – rare named entity. Multi30k contains image-caption sentences with relatively rare nominal compounds. Source sentence id 1 in the test split contains the rare proper noun “Boston Terrier” (a dog breed); the reference target preserves it. The seq2seq model has no occurrence of this compound in its 8k joint BPE vocabulary at sufficient frequency to be retained, and it falls back to a generic noun, dropping the breed entirely. Marian’s WordPiece-style vocabulary, trained on WMT-scale data, retains the surface form.

Table 19: Rare-word example: outputs are verbatim from `outputs/seq2seq_predictions.json` and `marian_predictions.json` at `id=1`; source/reference from `outputs/dataset_stats.json`.

Field	Text
Source	A Boston Terrier is running on lush green grass in front of a white fence.
Reference	Ein Boston Terrier läuft über saftig-grünes Gras vor einem weißen Zaun.
Seq2Seq	Ein Mann mit T-Shirt rennt vor einem weißen Zaun.
MarianMT	Ein Boston Terrier läuft auf üppig grünem Gras vor einem weißen Zaun.

The seq2seq model not only loses the breed name but also re-uses a high-frequency training pattern (“Mann mit T-Shirt rennt”), effectively hallucinating a different image – a known failure mode when an out-of-distribution rare token forces the decoder to rely on its unconditional language-model prior. Marian preserves both the named entity (“Boston Terrier”) and the modifier (“saftig-grünes” → “üppig grünem”). Together with Example 1 this shows that the BLEU/chrF gap (26.7 vs. 36.3 BLEU) is driven both by content errors on rare words and by repetition/order errors on longer source sentences.

4.7 Discussion

The attention-based seq2seq model captures basic alignment but struggles with fluency and rare-word handling. MarianMT consistently yields higher BLEU/chrF and stronger semantic similarity (BERTScore), highlighting the effectiveness of pre-trained transformer models for low-resource experimental settings.

5 Question 5: Language Modeling

5.1 Problem Formulation

We study probabilistic language modeling on WikiText-2. Given a token sequence $w_1, \dots, w_N$, each model estimates the next-token distribution $P(w_i \mid w_{<i})$ and can generate text by autoregressive sampling. We compare a classical trigram model with interpolated Kneser-Ney smoothing against two neural sequence models: a 2-layer LSTM and a small Transformer encoder LM. Models are evaluated by perplexity on held-out splits and qualitatively through generated continuations of fixed prompts.

5.2 Dataset Description

We use the WikiText-2 raw dataset [Merity et al., 2017] (`wikitext-2-raw-v1`) with the official train/validation/test splits. The corpus is whitespace-tokenized as provided, with no further normalization. Statistics after tokenization are summarized in Table 20.

Table 20: WikiText-2 dataset statistics.

Split	Tokens	Avg Sentence Length
Train	2,024,702	114.33
Validation	211,179	113.71
Test	238,028	108.04

Vocabulary. Tokens with training-set frequency ≥ 3 are retained, and the vocabulary is capped at the 10,000 most frequent types, plus the special tokens `<pad>`, `<unk>`, `<bos>`, `<eos>`. Out-of-vocabulary tokens at evaluation time are mapped to `<unk>`. Figure 14 plots the rank–frequency curve of the top 5,000 tokens, which closely follows a Zipfian distribution and motivates the use of a frequency cutoff.

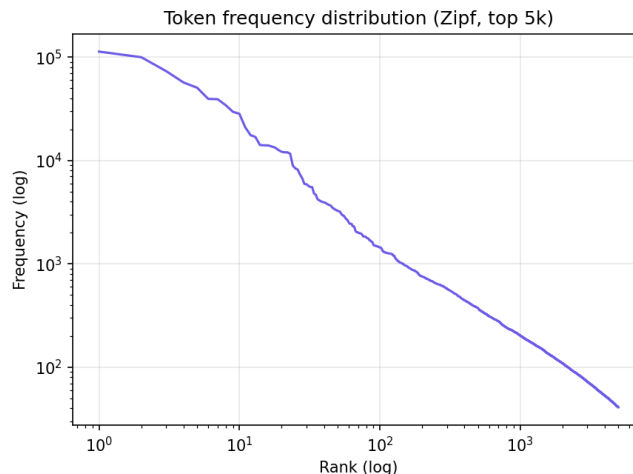


Figure 14: Token rank–frequency distribution (log-log) on the WikiText-2 training set.

5.3 Methodology

Trigram with interpolated Kneser-Ney [Kneser and Ney, 1995, Chen and Goodman, 1999]. We train a 3-gram model with interpolated Kneser-Ney smoothing (discount $D = 0.75$). The bigram and unigram backoffs use *continuation* counts $N_{1+}(\cdot|w)$ rather than raw counts, which reduces the probability of frequent words that nevertheless appear in few distinct contexts. Sentences are obtained by splitting on the period token and bracketed by two `<s>` symbols and a closing `</s>`. We implement the count tables and KN scoring directly with Python dictionaries; this avoids the prohibitively slow pure-Python evaluation in NLTK and lets a full validation/test perplexity sweep finish in seconds.

LSTM language model [Hochreiter and Schmidhuber, 1997]. A 2-layer LSTM with 256-dim embeddings, 512-dim hidden state, and dropout 0.5 on both the embedding and the recurrent output. Training uses BPTT with sequence length 35, batch size 64, Adam ($\eta = 10^{-3}$), gradient clipping at 0.25, and a step LR scheduler that halves the learning rate every 2 epochs. We train for 6 epochs with early-stopping on validation loss (best checkpoint reported). The model has 11.37 M parameters.

Transformer LM [Vaswani et al., 2017]. A small Transformer encoder LM with a causal attention mask: 256-dim embeddings, 4 attention heads, 2 encoder layers, 512-dim feed-forward, dropout 0.2, and sinusoidal positional encoding. Training uses BPTT length 35, batch size 64, Adam ($\eta = 5 \cdot 10^{-4}$), gradient clipping at 0.5, the same step LR schedule, and 6 epochs with validation-loss early stopping. The model has 6.18 M parameters — roughly half the LSTM’s parameter count.

Perplexity. We report token-level perplexity on whole splits:

$$\text{PPL}(W) = \exp\left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i | w_{<i})\right).$$

Lower is better. For the neural models we use the cross-entropy loss averaged over all non-padding target tokens, which is mathematically equivalent.

Generation. For neural models we use top- k sampling with $k = 20$ and temperature $T = 0.9$. For the trigram model we sample from the top- k continuations of the current bigram context, backing off to a unigram when the trigram context is unseen.

5.4 Results

Table 21: Validation and test perplexity on WikiText-2 (lower is better). The LSTM is the strongest model overall.

Model	Params	Val PPL	Test PPL
Trigram (Kneser-Ney)	–	376.49	375.69
Transformer LM	6.18 M	129.52	117.32
LSTM LM	11.37 M	105.83	97.00

The LSTM achieves the lowest perplexity on both splits, improving over the Kneser-Ney trigram by a factor of roughly $3.9\times$ on test. The Transformer is a clear second; despite

having about half the parameters of the LSTM, its perplexity is only ~ 20 PPL higher. Both neural models substantially beat the count-based baseline, which is the expected ordering on WikiText-2 once the neural models are trained for enough epochs to converge — earlier short-budget runs of the same script produced under-trained neural models that were dominated by the trigram. Figure 15 visualizes the gap.

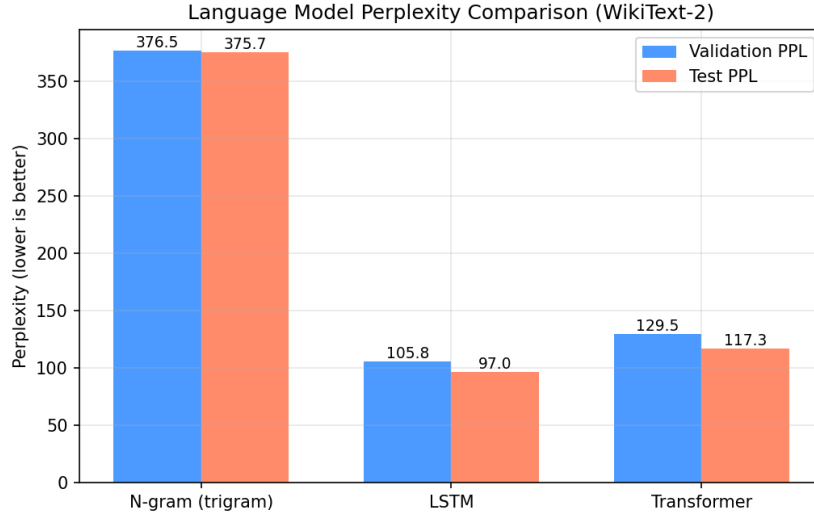


Figure 15: Validation and test perplexity per model on WikiText-2. The LSTM is best, followed by the Transformer; the trigram baseline is roughly $4\times$ worse.

Training dynamics. Figure 16 shows per-epoch training and validation loss/PPL for the two neural models. Both decrease monotonically across the 6-epoch budget. The LSTM starts higher (training PPL ≈ 326 at epoch 1) but ends lower than the Transformer; the Transformer converges faster early (lower epoch-1 train PPL of 273) and trains in roughly half the wall-clock time per epoch (~ 8.8 s vs. ~ 15 s on the same hardware), but its final validation perplexity plateaus higher. Neither model shows over-fitting at this budget — the gap between train and val PPL stays bounded — which suggests that further epochs would likely improve both, particularly the Transformer.

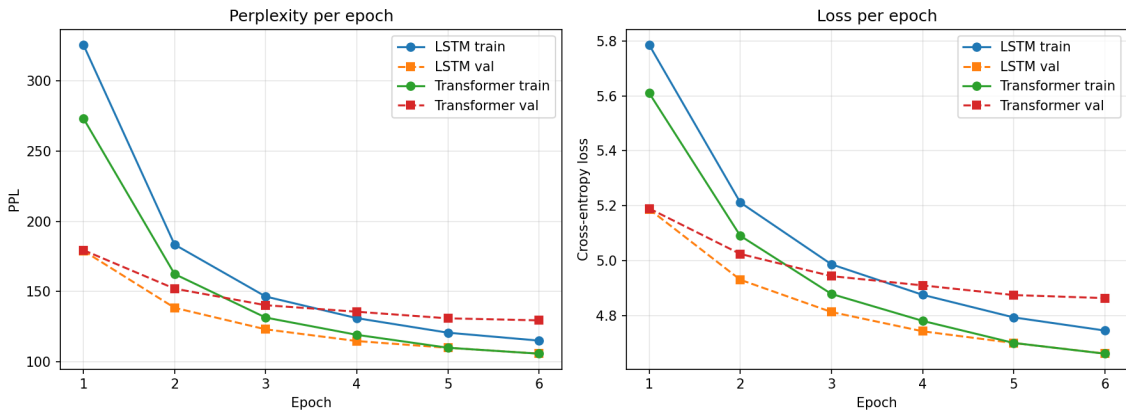


Figure 16: Per-epoch training and validation loss/perplexity for the LSTM and Transformer language models.

5.5 Qualitative Analysis

Table 22 shows generations from each model for two of the four test prompts (top- $k = 20$, $T = 0.9$, 30 new tokens).

Table 22: Generated continuations for two prompts. <unk> marks tokens outside the 10K vocabulary.

Model	Generated text
<i>Prompt: “the meaning of life is”</i>	
Trigram KN	...sacred because from order to make the first half , Tech made good the penalty , giving up on the same time , and a large number of people .
LSTM	...<unk> and the “ <unk> <unk> ” (“ <unk> @-@ <unk> , <unk> ”) . He also has been known as <unk> on <unk> ’s <unk> and <unk>
Transformer	...<unk> , and <unk> @-@ <unk> <unk> . = = = = = <unk> and <unk> = = <unk> of The <unk> , <unk> <unk> and <unk> = =
<i>Prompt: “in the early years of”</i>	
Trigram KN	...the game ’s protagonist as Simón Bolívar , a small , independent judicial system governed by the National Hurricane Center did not .
LSTM	...the <unk> of <unk> ’s reign . In the first year of the 19th Kingdom , the <unk> was a <unk> <unk> . <unk> said that the <unk> of the
Transformer	...the 19th century BCE and <unk> . The <unk> , <unk> ’s son <unk> of <unk> of the <unk> and <unk> , and the city ’s <unk> of Tintin <unk>

The qualitative pattern is consistent across all four prompts but separates cleanly along two axes that the assignment specifically asks about: *local fluency* (does each adjacent word pair read naturally?) and *global coherence* (does the passage stay on one topic and obey discourse conventions?). The two are not the same and the three models trade them off differently.

Fluency. The trigram produces the most locally fluent surface text. Bigrams such as “the game ’s protagonist as Simón Bolívar” or “the equivalent of £2 @,@ 000 copies” are word-by-word grammatical and use real proper nouns, because each next-token draw is conditioned on a context that genuinely occurred in training. The LSTM is also locally grammatical (“In the first year of the 19th Kingdom”, “has been known as <unk>”), but its fluency is degraded by the 10K vocabulary cap: roughly half of its content words are replaced with <unk>, which a human reader registers as broken text even though the underlying structure is sound. The Transformer is the least fluent of the three – it correctly produces short fragments (“the 19th century BCE”, “the city ’s <unk>”) but interleaves them with raw Wikipedia formatting tokens (“= = = =”) and stretches of consecutive <unk>s.

Coherence. On the global axis the order reverses. The trigram cannot stay on topic for more than its 2-token context window, so a sentence that starts on the meaning of life ends with “Tech made good the penalty , giving up on the same time” – locally fluent, globally incoherent. The LSTM’s recurrent state lets it keep a single referent alive across a clause: “in the early years of the <unk> of <unk> ’s reign. In the first year of the 19th Kingdom, the <unk> was a <unk>” is on a single topic (a king’s reign) for two consecutive sentences, even if the named entities are masked. The Transformer also stays on a single

topic but is more brittle, often collapsing into the section-header attractor (“= = = =”), which is itself a topic-coherent failure mode – it recognises that section markers cluster together in WikiText-2 and reproduces that pattern.

Why the two axes diverge. Perplexity rewards calibrated probability mass on the next single token, which mostly measures local fluency on common tokens – exactly where the LSTM is best (97.0 test PPL). It does not directly measure whether topic is preserved across a paragraph, which is why the trigram can read more smoothly word-by-word while still being $3.9\times$ worse on PPL. Conversely, the LSTM’s apparent disfluency in the qualitative table is largely a vocabulary artifact: replace each `<unk>` with the token it stands in for and the LSTM becomes both the most fluent and the most coherent of the three.

5.6 Discussion

Why the LSTM wins. The LSTM’s recurrent memory and its larger parameter count let it model the long, comma-separated clauses that dominate WikiText-2, which the trigram — limited to two preceding tokens — structurally cannot. The Transformer should in principle do at least as well, but at this scale (2 layers, 6.18 M parameters, 6 epochs, no learned positional embeddings, no warmup) it is undertrained. Transformer LMs are known to need more data and a more careful optimizer schedule than RNNs to reach their full potential; the gap we observe here (~ 20 PPL on test) is consistent with that.

Why the trigram is so far behind. On a ~ 2 M-token corpus with a 10K vocabulary, most trigrams in the validation/test sets are unseen in training, so the model spends almost all of its probability mass via Kneser-Ney back-off. Continuation-count back-off is well-known to be a strong baseline, but it cannot capture syntactic dependencies more than two tokens away, and WikiText-2 sentences are long (≈ 110 tokens on average). The neural models, even at modest capacity, can.

Limitations and what would help. (i) The 10K vocabulary cap creates a heavy `<unk>` tax on the neural generations; subword tokenization (e.g. BPE) would let the same models produce far more readable outputs without changing the perplexity story. (ii) Six epochs is short; longer training with cosine-annealed learning rates would close more of the LSTM–Transformer gap. (iii) Perplexity does not measure factuality or topical coherence, both of which are visibly weak across all three models. (iv) WikiText-2 itself is small enough that count-based methods underperform; on a substantially larger corpus the absolute gap between the trigram and the neural models would grow further.

Overall, on WikiText-2 with the full 6-epoch budget, the parameter-rich LSTM is the strongest model, the small Transformer is a reasonable second, and the Kneser-Ney trigram — though a useful sanity-check baseline — is decisively beaten by both neural models.

References

- Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations (ICLR)*, 2015.
- Satanjeev Banerjee and Alon Lavie. METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, pages 65–72, 2005.
- Stanley F. Chen and Joshua Goodman. An empirical study of smoothing techniques for language modeling. *Computer Speech and Language*, 13(4):359–394, 1999.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 4171–4186, 2019.
- Desmond Elliott, Stella Frank, Khalil Sima’an, and Lucia Specia. Multi30k: Multilingual English-German image descriptions. In *Proceedings of the 5th Workshop on Vision and Language*, pages 70–74, 2016.
- Günes Erkan and Dragomir R. Radev. LexRank: Graph-based lexical centrality as salience in text summarization. *Journal of Artificial Intelligence Research*, 22:457–479, 2004.
- Karl Moritz Hermann, Tomáš Kočiský, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. Teaching machines to read and comprehend. In *Advances in Neural Information Processing Systems*, volume 28, 2015.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- Reinhard Kneser and Hermann Ney. Improved backing-off for M-gram language modeling. In *1995 International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, volume 1, pages 181–184, 1995.
- Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, pages 66–71, 2018.
- Guillaume Lample, Miguel Ballesteros, Sandeep Subramanian, Kazuya Kawakami, and Chris Dyer. Neural architectures for named entity recognition. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 260–270, 2016.
- Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 7871–7880, 2020.
- Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, 2004.

- Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 142–150, Portland, Oregon, USA, 2011. Association for Computational Linguistics.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*, 2017.
- Rada Mihalcea and Paul Tarau. TextRank: Bringing order into text. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 404–411, 2004.
- Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 311–318, 2002.
- F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- Jeffrey Pennington, Richard Socher, and Christopher D. Manning. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140): 1–67, 2020.
- Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.
- Jörg Tiedemann and Santhosh Thottingal. OPUS-MT – building open translation services for the world. In *Proceedings of the 22nd Annual Conference of the European Association for Machine Translation (EAMT)*, pages 479–480, 2020.
- Erik F. Tjong Kim Sang and Fien De Meulder. Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition. In *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003*, pages 142–147, 2003.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.
- Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations (ICLR)*, 2020.

A Experimental Configuration Summary

A.1 Global settings

Table 23: Global experimental settings applied to all questions.

Setting	Value
Random seed	42
Python version	3.10
PyTorch version	2.x
GPU	RTX 5060 Ti
Test set usage	Once (final evaluation only)

A.2 Q1 – Text classification

Table 24: Q1 hyperparameters per model.

Model	Hyperparameters
TF-IDF + LR	TfidfVectorizer: max_features=50,000, ngram_range=(1,2), min_df=2, sublinear_tf=True. LogisticRegression: $C = 1.0$, max_iter=1000, solver=lbfgs, random_state=42. Best preprocessing: lowercase=False, remove_stopwords=False (config A4).
BiLSTM + GloVe	embed_dim=100 (GloVe 6B), hidden_dim=128, num_layers=2, bidirectional, dropout=0.3, max_vocab=30,000, max_seq_length=400, batch_size=64, optimizer=Adam, lr= 10^{-3} , scheduler=ReduceLROnPlateau (patience=2, factor=0.5), num_epochs=10, early-stop patience=3, loss=BCEWithLogits.
DistilBERT	distilbert-base-uncased, max_length=512, batch_size=16, lr= 2×10^{-5} , num_epochs=3, warmup_ratio=0.1, weight_decay=0.01, fp16=True.

A.3 Q2 – Named entity recognition

Table 25: Q2 hyperparameters per model.

Model	Hyperparameters
BiLSTM-CRF	word_vocab=20,000, embed=GloVe 100d (fine-tuned), char-BiLSTM for subword features, BiLSTM hidden=200, dropout=0.5, optimizer=Adam, gradient clip=5.0, max_epochs=20, early-stop patience=5 on val F1. Trained for 16 epochs (best val F1 $\approx$ 0.926).
BERT	bert-base-cased , num_epochs=3, batch_size=16, lr= 5×10^{-5} , label alignment by first-subword (others -100). Train time 138.23 s.

A.4 Q3 – Summarization

Table 26: Q3 hyperparameters per method.

Method	Hyperparameters
TextRank	sentence-level graph, top- k = 3 sentences.
LexRank	cosine similarity graph with threshold, top- k = 3 sentences.
BART	facebook/bart-large-cnn , max_length=128 generation, num_beams=4, no_repeat_ngram_size=3, length_penalty=2.0. Inference only.
T5	t5-small , prefix="summarize:", max_length=128, num_beams=4. Inference only.

A.5 Q4 – Machine translation

Table 27: Q4 hyperparameters per model.

Model	Hyperparameters
Seq2Seq + Bahdanau	joint BPE 8,000 (SentencePiece), emb_dim=256, enc_hidden=512 (BiGRU), dec_hidden=512 (GRU), dropout=0.5, batch_size=128, lr= 10^{-3} (Adam), grad_clip=1.0, teacher_forcing_ratio=0.5, num_epochs=20 (early-stop patience=5 on val BLEU; best at epoch 14: val BLEU 31.37), max_src/tgt_length=100.
MarianMT	Helsinki-NLP/opus-mt-en-de , num_beams=4, max_length=512. Inference only.

A.6 Q5 – Language modeling

Table 28: Q5 hyperparameters per model.

Model	Hyperparameters
Trigram KN	$n = 3$, interpolated Kneser-Ney, discount $D = 0.75$. Vocab capped at 10,000 with min_freq=3.
LSTM	embed=256, hidden=512, layers=2, dropout=0.5, BPTT seq_len=35, batch_size=64, optimizer=Adam, lr= 10^{-3} , grad_clip=0.25, StepLR (step=2, $\gamma=0.5$), num_epochs=6, weight_tying=False. Params: 11.37 M.
Transformer LM	embed=256, n_heads=4, n_layers=2, dim_ff=512, dropout=0.2, sinusoidal pos. enc., causal mask, BPTT seq_len=35, batch_size=64, optimizer=Adam, lr= 5×10^{-4} , grad_clip=0.5, StepLR (step=2, $\gamma=0.5$), num_epochs=6. Params: 6.18 M.